LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 19

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, RESTful API menjadi penghubung utama antara backend dengan berbagai jenis client seperti aplikasi web, aplikasi mobile, maupun sistem eksternal. Karena seluruh komunikasi data melewati API, kesalahan pada API akan berdampak langsung pada seluruh client yang memakainya. Oleh sebab itu, API perlu diuji sejak tahap pengembangan agar kesalahan dapat ditemukan lebih awal dan kualitas perangkat lunak meningkat.

Pada acara-acara sebelumnya, RESTful API Sistem Informasi Akademik telah dibangun menggunakan PHP Native dengan arsitektur MVC, lengkap dengan Front Controller, Router, dan class Response yang menyeragamkan format JSON serta kode status HTTP. Pada Acara 19 ini, pengujian API dilakukan secara lebih terstruktur menggunakan Postman, yaitu dengan membuat Collection, Environment, dan dokumentasi contoh response. Selain itu, API dikembangkan dengan resource baru berupa registrasi pengguna (autentikasi), yang mencakup tabel users, Model User, Controller AuthController, dan pemetaan baru pada Router. Kata sandi pengguna disimpan dalam bentuk hash menggunakan password_hash() sehingga tidak tersimpan sebagai teks biasa.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah sebagai berikut.

1. Memahami konsep dan aspek pengujian API (API Testing).
2. Menggunakan Postman untuk membuat Collection dan Environment serta menguji endpoint RESTful API.
3. Membuat tabel users dan menyimpan kata sandi secara aman menggunakan password_hash().
4. Menambahkan Model User, Controller AuthController, dan pemetaan register pada Router dalam arsitektur MVC yang sudah berjalan.
5. Menambahkan validasi format email menggunakan filter_var() dengan FILTER_VALIDATE_EMAIL.
6. Mendokumentasikan hasil pengujian sukses dan gagal pada Postman, serta membuat Environment kedua untuk simulasi lingkungan staging.

BAB 2. DASAR TEORI

2.1 Pengujian API (API Testing)

API Testing adalah proses pengujian terhadap Application Programming Interface (API) untuk memastikan setiap endpoint berfungsi sesuai spesifikasi yang dirancang. Berbeda dengan pengujian antarmuka pengguna (User Interface Testing), API Testing berfokus pada komunikasi antara client dan server, meliputi validitas request, struktur response, keakuratan data, performa, dan penanganan kesalahan.

Beberapa aspek yang perlu diperhatikan dalam pengujian API adalah:

- validitas endpoint;
- metode HTTP yang digunakan;
- parameter request;
- struktur data JSON;
- waktu respons (response time);
- kode status HTTP (HTTP Status Code);
- mekanisme autentikasi dan otorisasi.

2.2 Postman

Postman adalah aplikasi yang digunakan untuk mengembangkan, menguji, mendokumentasikan, dan memantau API. Postman menyediakan antarmuka grafis sehingga pengembang dapat mengirim HTTP request tanpa harus membuat aplikasi client terlebih dahulu. Pengguna dapat mengirim request GET, POST, PUT, DELETE, maupun PATCH, serta melihat response server, terutama dalam format JSON.

Fitur utama Postman meliputi:

- pengiriman HTTP request;
- pengaturan header;
- pengiriman parameter URL;
- pengiriman data menggunakan Body (JSON atau Form Data);
- penyimpanan koleksi (Collection);
- otomatisasi pengujian sederhana;
- dokumentasi API.

2.3 Komponen HTTP Request pada Postman

a. Request Method. Metode HTTP menunjukkan operasi yang dilakukan terhadap suatu resource.

Method	Fungsi
GET	Mengambil data
POST	Menambahkan data baru
PUT	Memperbarui seluruh data
PATCH	Memperbarui sebagian data
DELETE	Menghapus data

b. URL Endpoint. Endpoint adalah alamat resource yang diakses, misalnya http://localhost/Web_Server/Acara19/api/mahasiswa.php atau http://localhost/Web_Server/Acara19/api/mahasiswa.php?id=10.

c. Header. Header berisi informasi tambahan tentang request, misalnya Content-Type: application/json, Accept: application/json, dan Authorization: Bearer xxxxx. Header memberi tahu server mengenai format data yang dikirim dan mekanisme autentikasi yang digunakan.

d. Body. Body digunakan untuk mengirim data ke server, terutama pada metode POST dan PUT. Contoh Body JSON:

```
{
  "nama": "Andi",
  "prodi": "Teknik Informatika"
}
```

2.4 Response pada Postman

Setelah memproses request, server mengembalikan response yang terdiri atas HTTP Status Code, Response Header, Response Body, Response Time, dan Response Size. Contoh response:

```
{
  "status": "success",
  "message": "Data berhasil ditambahkan."
}
```

Response tersebut menunjukkan bahwa server berhasil memproses permintaan client.

2.5 Konsep Dasar Autentikasi pada REST API

Terdapat beberapa skema umum autentikasi pada REST API.

- **Basic Authentication** mengirimkan kredensial username dan password yang di-encode Base64 pada setiap request.
- **API Key** adalah token unik yang diberikan kepada klien dan disertakan pada header atau parameter request.
- **Bearer Token (termasuk JWT)** adalah token yang diperoleh setelah login berhasil dan disertakan pada header Authorization pada setiap request berikutnya.

Pemilihan skema bergantung pada kebutuhan aplikasi. Untuk aplikasi dengan satu jenis klien yang dikendalikan sendiri oleh pengembang, skema token sederhana yang stateless seperti Bearer Token umumnya sudah memadai dan lebih ringan dibandingkan OAuth, yang dirancang untuk banyak aplikasi pihak ketiga.

2.6 Tabel Pengguna dan Penyimpanan Kata Sandi

Sebelum autentikasi diterapkan, dibutuhkan tabel users yang menyimpan identitas pengguna. Kata sandi tidak boleh disimpan dalam bentuk teks biasa (plain text). Berdasarkan dokumentasi resmi PHP Manual, fungsi password_hash() dengan algoritma PASSWORD_DEFAULT direkomendasikan untuk menghasilkan hash kata sandi yang aman, dan fungsi password_verify() digunakan untuk mencocokkan kata sandi yang dimasukkan pengguna saat login dengan hash yang tersimpan.

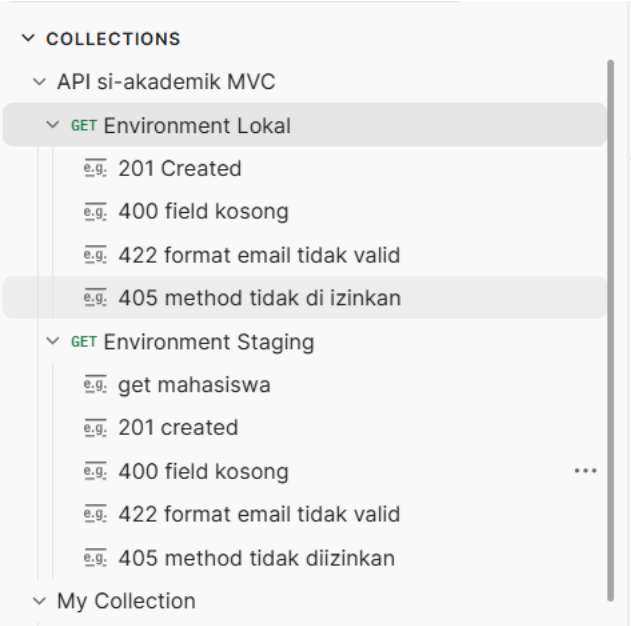
BAB 3. HASIL PRAKTIKUM

3.1 Prosedur Kerja

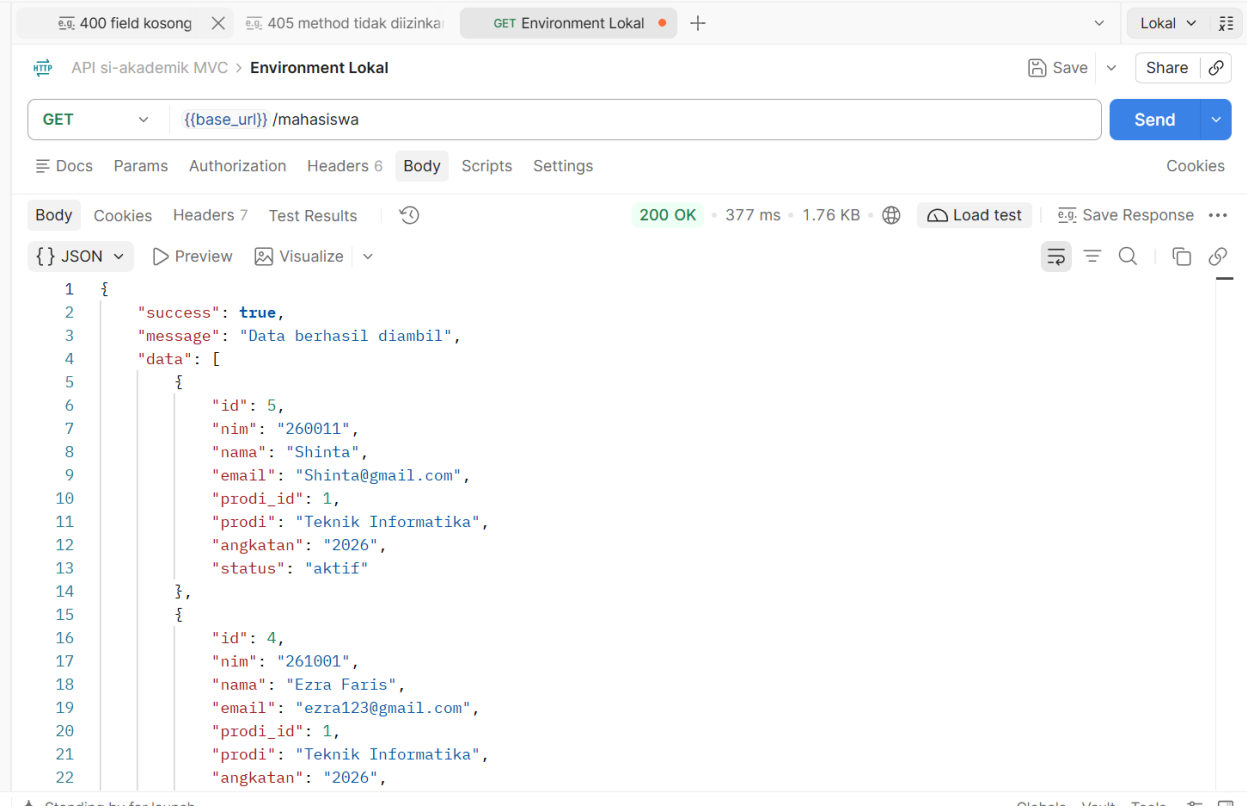
Prosedur kerja menambahkan resource baru (autentikasi) ke dalam arsitektur MVC yang sudah berjalan, yaitu Model User, Controller AuthController, dan pemetaan baru pada Router. Struktur project berada pada folder si-akademik dengan database si_akademik.

Langkah 1. Membuat Postman Collection dengan URL Baru

1. Membuat Collection baru bernama "API Sistem Informasi Akademik (MVC)".
2. Membuat Environment "Lokal" dengan variabel base_url bernilai http://localhost/Web_Server/Acara19/si-akademik/public.
3. Memindahkan request mahasiswa dari Acara 17 dan 18 ke Collection ini dengan URL {{base_url}}/mahasiswa.



Gambar 3.1 Collection API si-akademik MVC



Gambar 3.2 Environment Lokal dengan variabel base_url

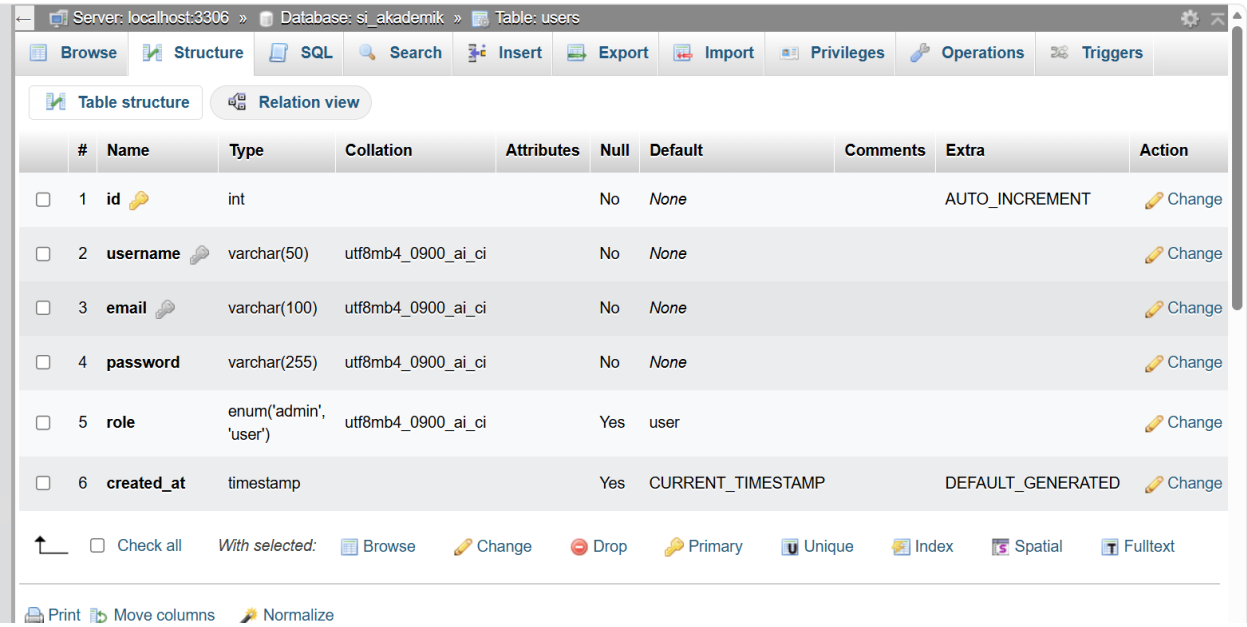
Hasil: variabel {{base_url}} terbaca oleh Postman dan request mahasiswa tetap berjalan dengan response 200 setelah dipindahkan ke Collection baru.

Langkah 2. Membuat Tabel users

Tabel users dibuat pada database si_akademik melalui phpMyAdmin dengan perintah SQL berikut.

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin','user') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Kolom password dibuat VARCHAR(255) karena hasil password_hash() cukup panjang. Kolom username dan email bersifat UNIQUE, dan kolom role bernilai default user.



Gambar 3.4 Struktur tabel users

Langkah 3. Membuat Model User

Model `User` dibuat pada file `app/models/User.php`. Model ini berisi method `create()` untuk menyimpan pengguna baru dan `findByUsername()` untuk mencari pengguna berdasarkan username (akan dipakai pada proses login di acara berikutnya). Method `isValidEmail()` merupakan tambahan dari Tugas Praktik Mandiri 1.

Acara19 > si-akademik > app > models > User.php > User

```php
<?php

2 references | 0 implementations
class User
{
    4 references
    private PDO $db;

    1 reference | 0 overrides
    public function __construct()
    {
        $this->db = Database::getConnection();
    }

    // Tugas Mandiri 1: validasi format email
    1 reference | 0 overrides
    public function isValidEmail(string $email): bool
    {
        return filter_var($email, FILTER_VALIDATE_EMAIL) !== false;
    }

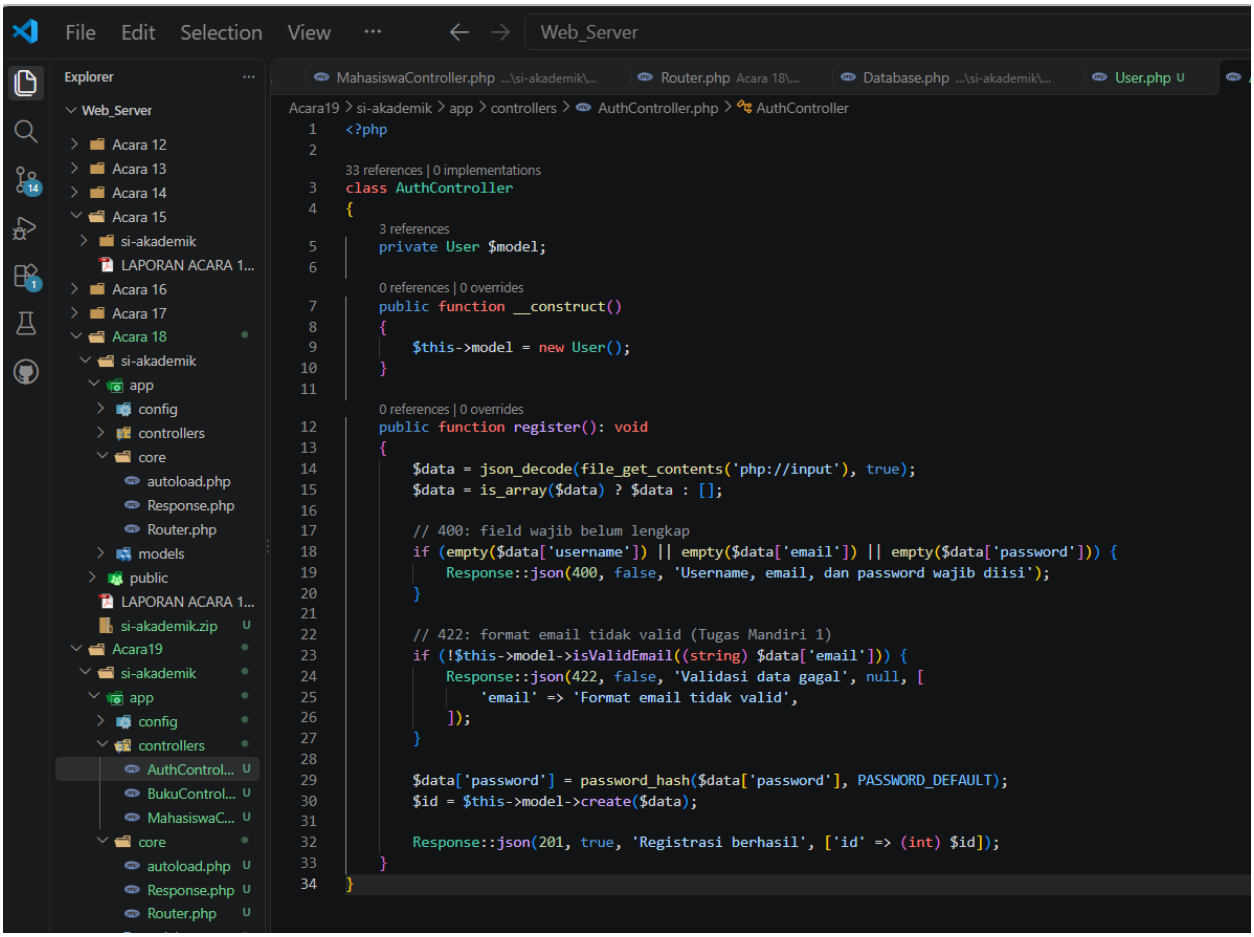
    1 reference | 0 overrides
    public function create(array $data): string
    {
        $stmt = $this->db->prepare(
            'INSERT INTO users (username, email, password) VALUES (:u, :e, :p)'
        );
        $stmt->execute([
            ':u' => $data['username'],
            ':e' => $data['email'],
            ':p' => $data['password'],
        ]);
        return $this->db->lastInsertId();
    }

    0 references | 0 overrides
    public function findByUsername(string $username)
    {
        $stmt = $this->db->prepare('SELECT * FROM users WHERE username = :u');
        $stmt->execute([':u' => $username]);
        return $stmt->fetch();
    }
}
```

Gambar 3.5 Kode Model User

Langkah 4. Membuat Controller AuthController

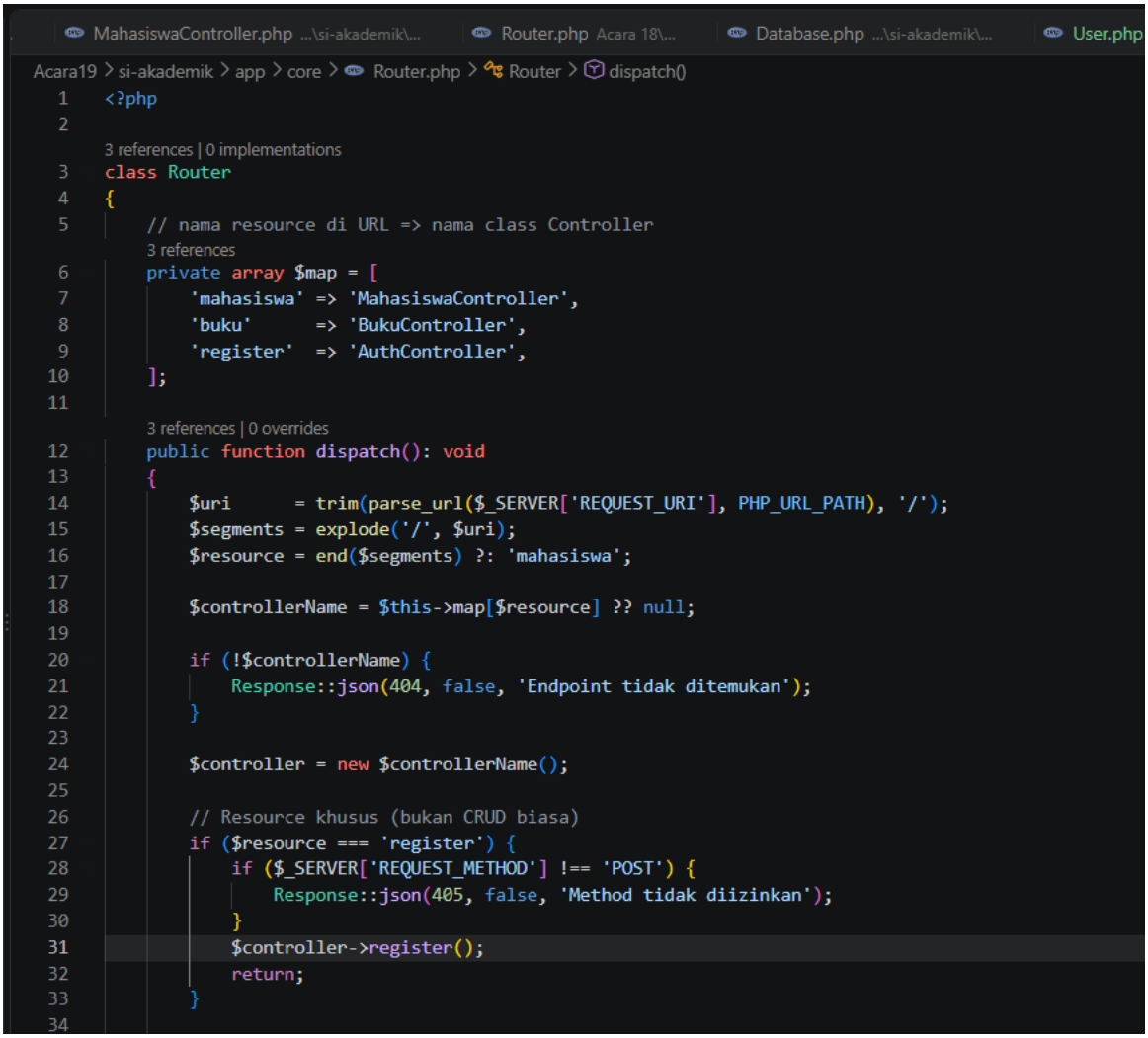
AuthController pada app/controllers/AuthController.php merupakan Controller pertama yang tidak sepenuhnya mengikuti pola CRUD standar (index, store, update, destroy), karena hanya berisi aksi tunggal register(). Aksi login() akan ditambahkan pada acara 20. Pada register(), data dibaca dari body JSON, dicek kelengkapannya (400), dicek format emailnya (422), kemudian kata sandi di-hash dan data disimpan (201).



Gambar 3.6 Kode AuthController

Langkah 5. Mendaftarkan Resource register pada Router

Karena register bukan resource CRUD biasa, ditambahkan penanganan khusus pada Router.php, yaitu pemetaan 'register' => 'AuthController' dan blok khusus sebelum switch method HTTP. Pada blok tersebut ditambahkan pengecekan bahwa hanya method POST yang diizinkan.



Gambar 3.7 Perubahan pada Router

Langkah 6. Menguji dan Mendokumentasikan pada Collection

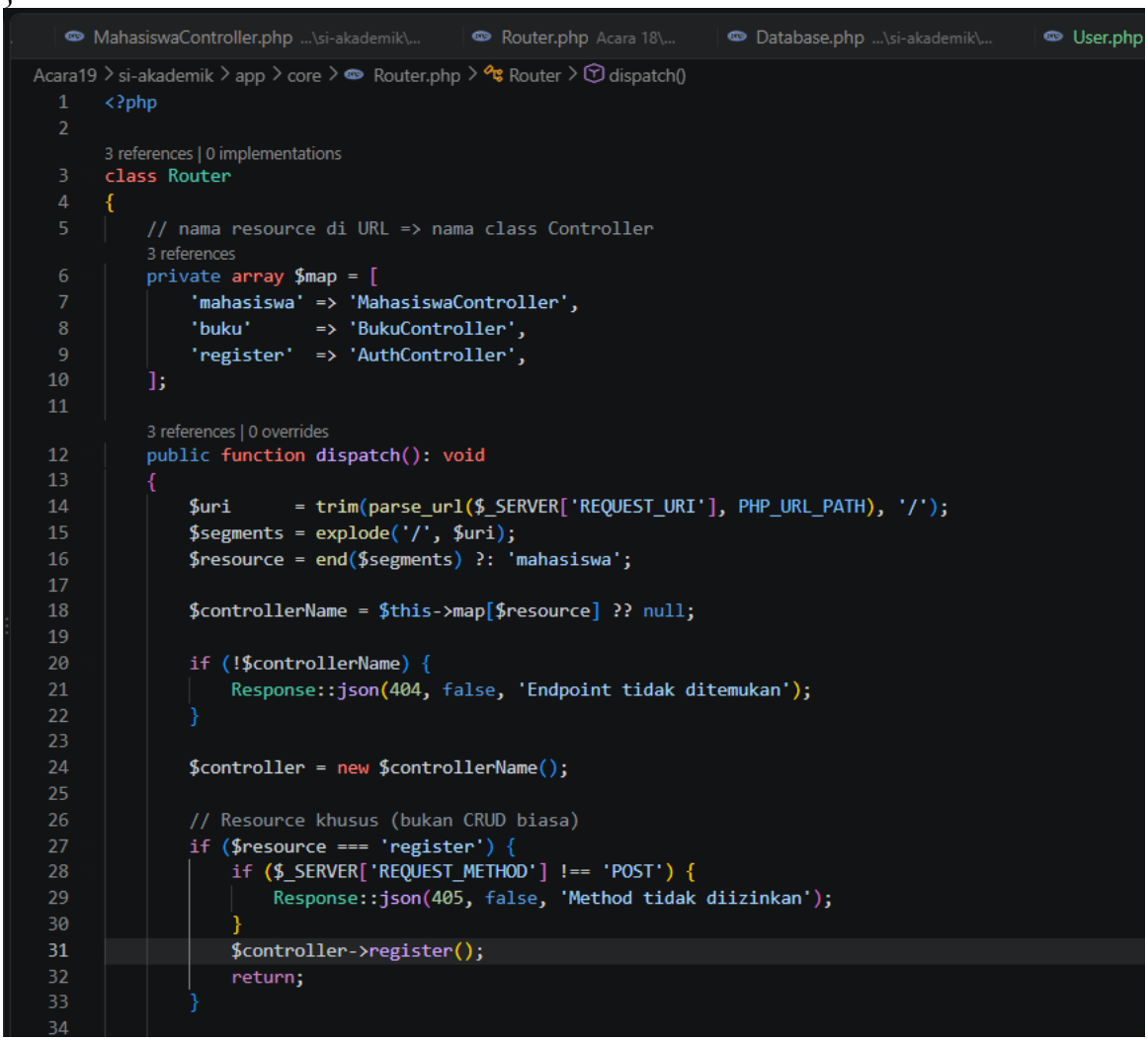
Request POST {{base_url}}/register ditambahkan ke Collection dengan Body bertipe raw JSON. Pengujian dilakukan dengan beberapa data percobaan, dan contoh response sukses maupun gagal disimpan pada masing-masing request sebagai dokumentasi awal (Save as example).

No	Skenario	Request	Hasil yang diharapkan
1	Registrasi berhasil	POST dengan username, email valid, dan password	201 Created, pesan "Registrasi berhasil" dan id pengguna
2	Field wajib kosong	POST tanpa field password	400 Bad Request
3	Format email tidak valid	POST dengan email budi-email	422 Unprocessable Entity, errors.email
4	Method tidak	GET ke /register	405 Method Not Allowed

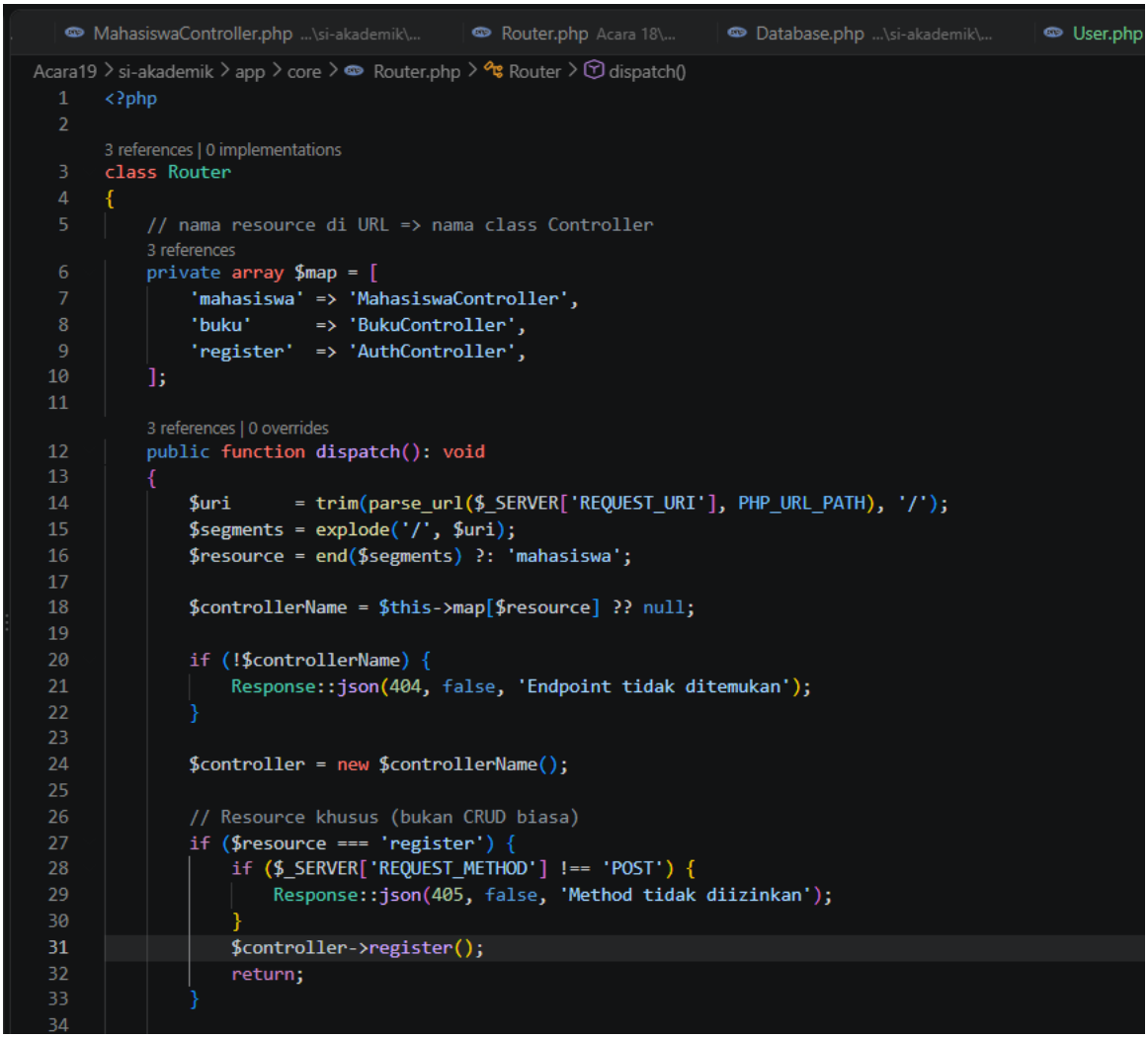
No	Skenario	Request	Hasil yang diharapkan
	diizinkan		

Contoh Body request sukses:

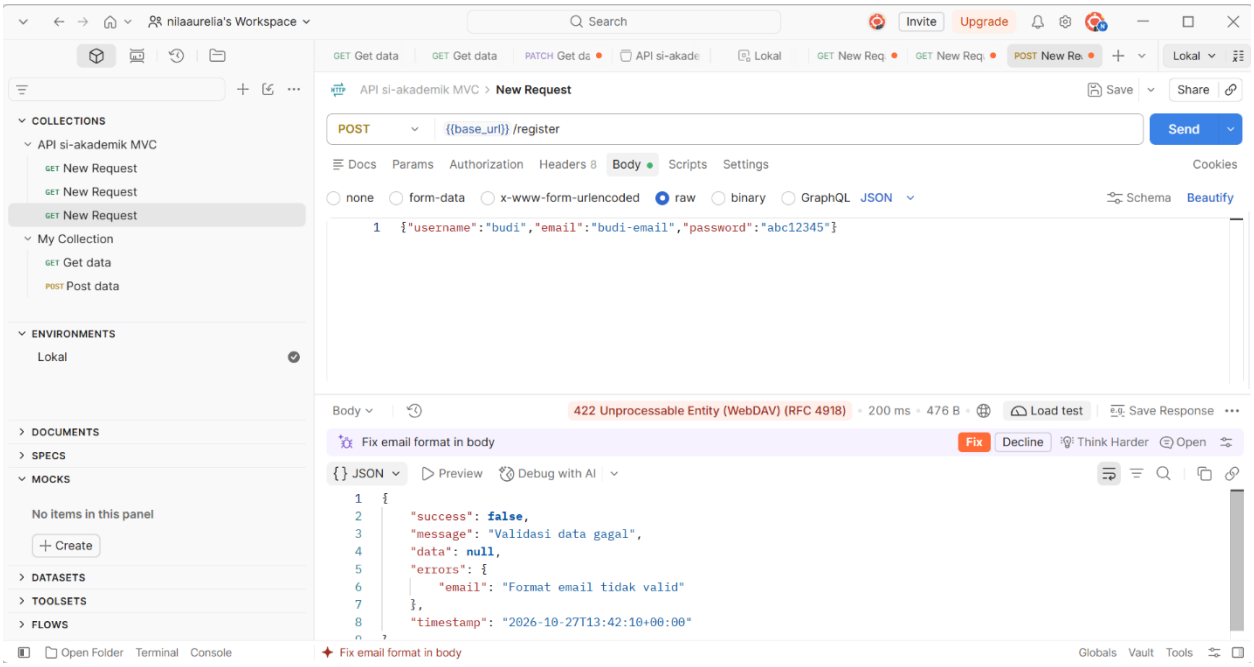
```
{
 "username": "nila",
 "email": "nila@email.com",
 "password": "rahasia123"
}
```



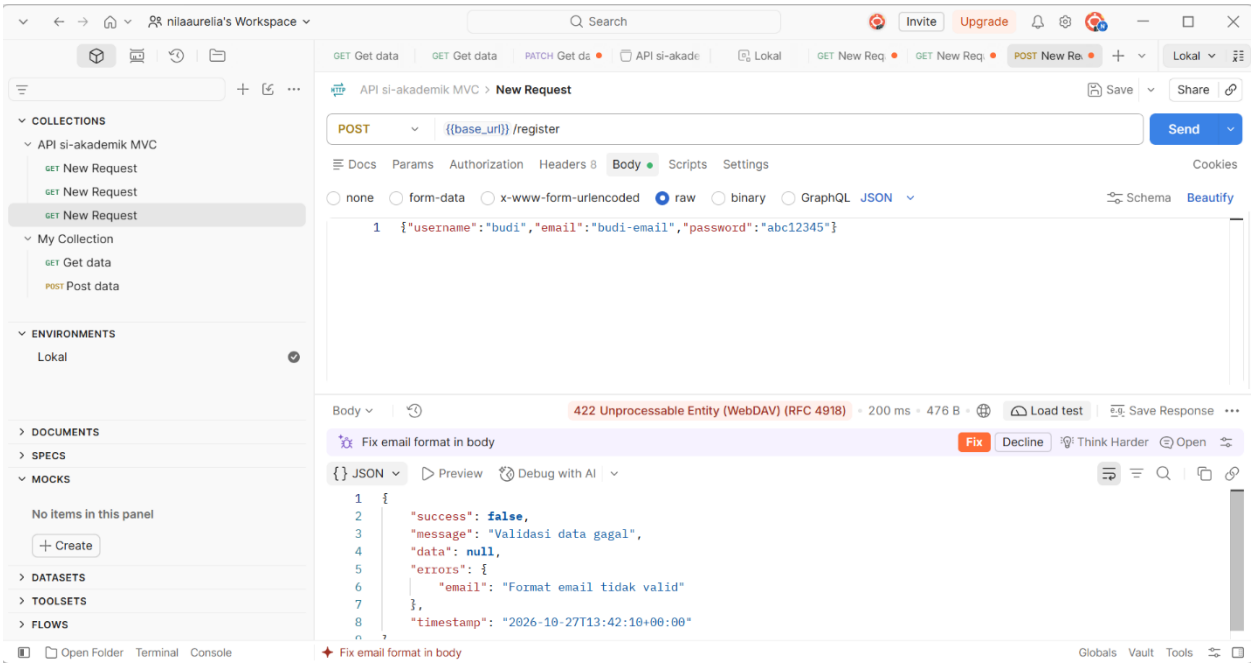
Gambar 3.8 Pengujian registrasi berhasil (201 Created)



Gambar 3.9 Pengujian field wajib kosong (400 Bad Request)



Gambar 3.10 Pengujian format email tidak valid (422)



]

Gambar 3.11 Pengujian method tidak diizinkan (405 Method Not Allowed)

	id	username	email	password	role	created_at
Edit Copy Delete	2	nila	nila@email.com	$2y$10$qSrw3x2TtBowiAxwAzzTrehKcHimdRfJnPCx3QRSdj/...	user	2026-10-27 20:46:43
Edit Copy Delete	3	ani	ani@email.com	$2y$10$VEr0haAR7/LbOHxngI4srOHgS8LoaFQZTve8zIs7MrJ...	user	2026-10-27 20:56:44

Gambar 3.12 Data pada tabel users, kata sandi tersimpan dalam bentuk hash

Pada tabel users, kolom password berisi hash berawalan $2y$10$... dan bukan teks asli, sehingga sesuai dengan ketentuan penyimpanan kata sandi yang aman.

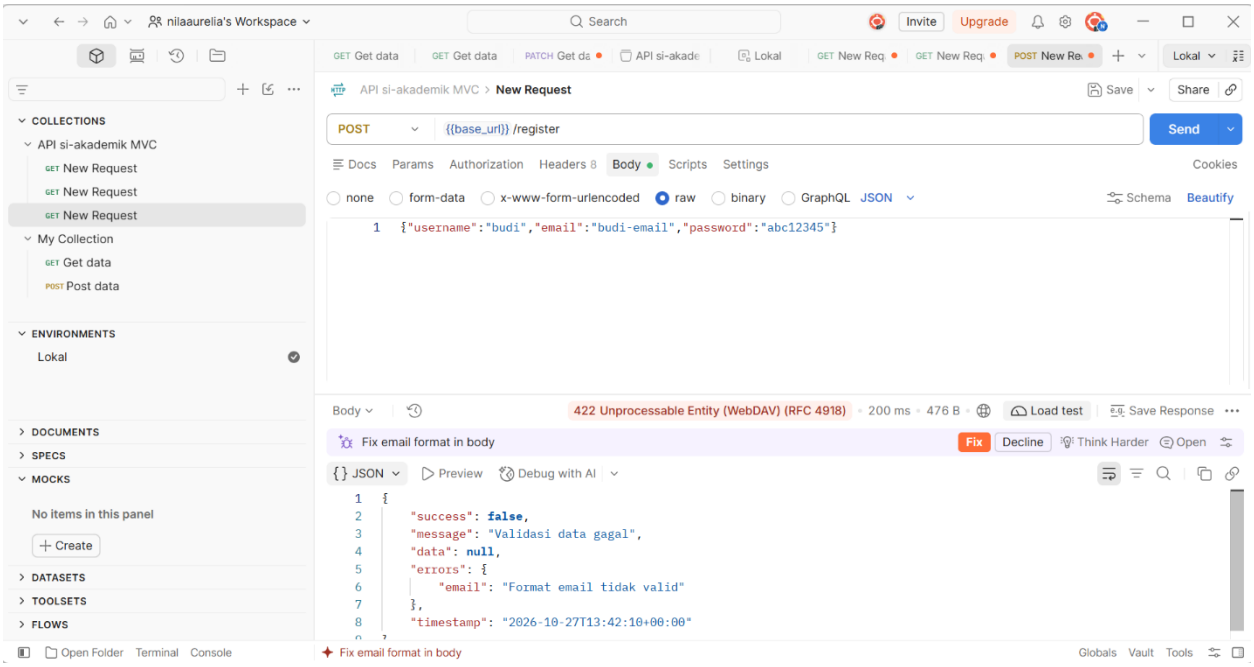
3.2 Tugas Praktik Mandiri

Tugas 1. Validasi Format Email pada Model User

Validasi format email ditambahkan pada Model User melalui method isValidEmail() yang memakai filter_var() dengan FILTER_VALIDATE_EMAIL. Method ini dipanggil oleh AuthController::register() sebelum data disimpan. Jika format email tidak valid, server mengembalikan status 422 dan data tidak masuk ke database.

```
public function isValidEmail(string $email): bool
{
   return filter_var($email, FILTER_VALIDATE_EMAIL) !== false;
}
```

Pengujian dilakukan dengan dua data, yaitu email tidak valid (budi-email) yang menghasilkan 422, dan email valid (budi@email.com) yang menghasilkan 201.



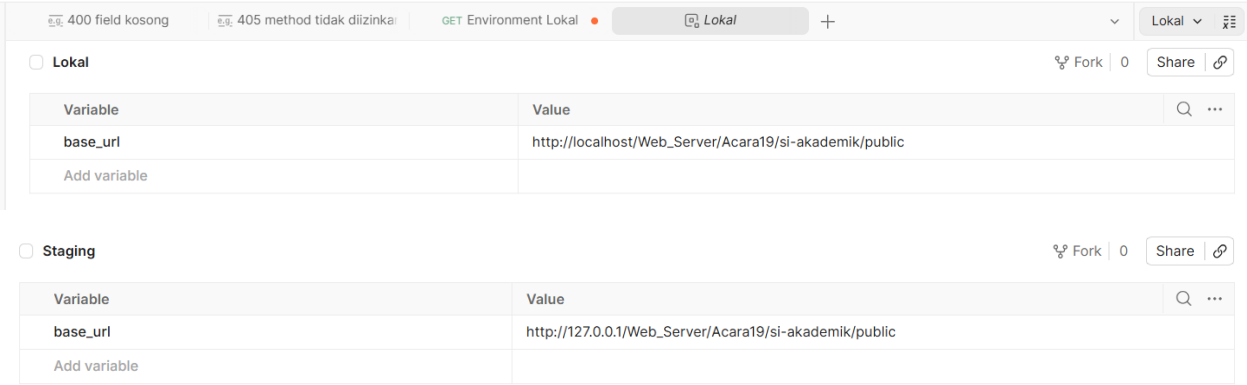
Gambar 3.13 Hasil pengujian validasi email

Tugas 2. Environment Kedua untuk Simulasi Staging

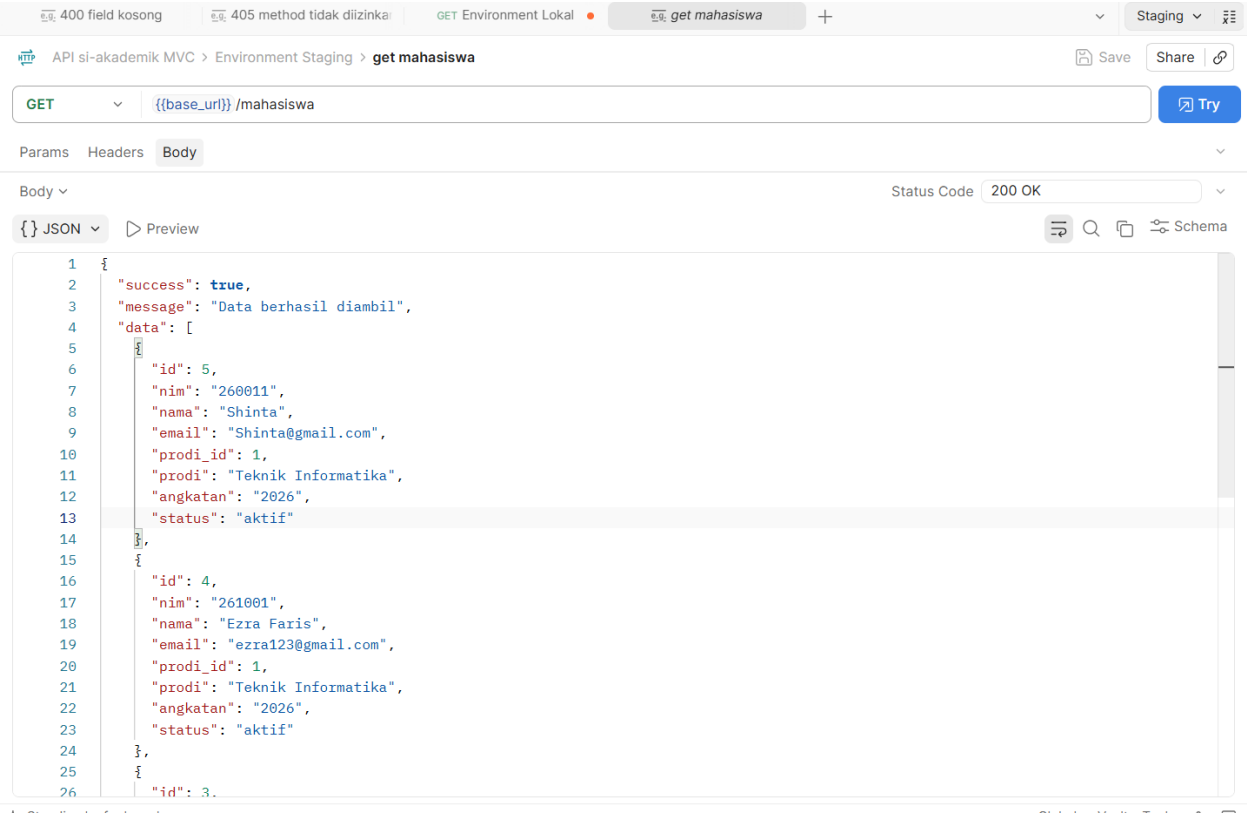
Dibuat Environment kedua bernama "Staging" dengan variabel base_url yang berbeda dari Environment Lokal untuk mensimulasikan lingkungan staging.

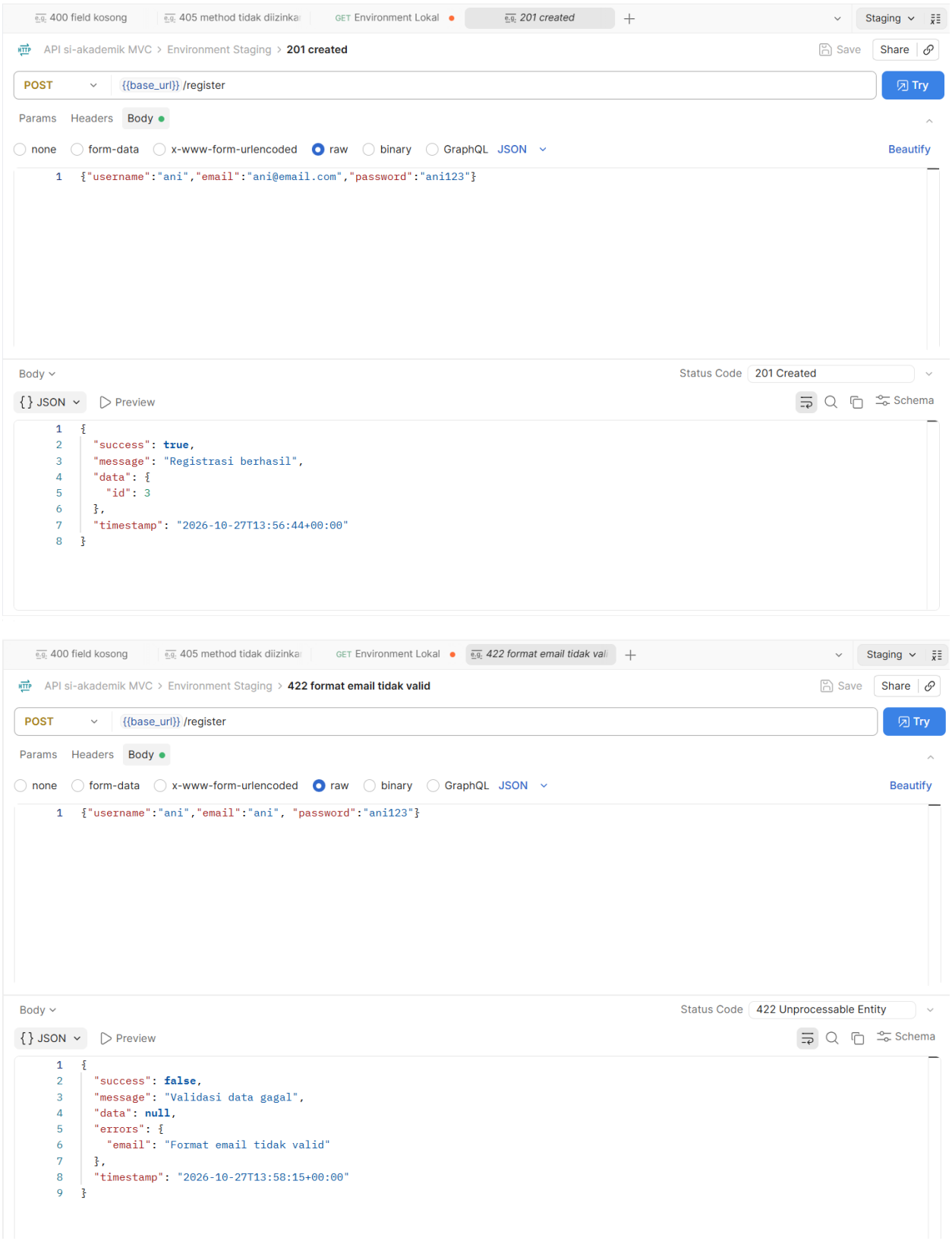
Environment	Nilai base_url
Lokal	http://localhost/Web_Server/Acara19/si-akademik/public
Staging	http://127.0.0.1/Web_Server/Acara19/si-akademik/public

Seluruh request pada Collection memakai {{base_url}}, sehingga untuk berpindah lingkungan cukup mengganti Environment pada dropdown di pojok kanan atas Postman tanpa mengubah satu pun request. Request yang sama dijalankan pada kedua Environment dan menghasilkan response yang sama.



Gambar 3.14 Environment Lokal dan Staging





Gambar 3.15 Pengujian request pada Environment Staging

BAB 4. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, Postman terbukti dapat digunakan untuk menguji RESTful API tanpa harus membuat aplikasi client. Dengan Collection dan Environment, seluruh request dapat dikelola dalam satu tempat, dan penggunaan variabel {{base_url}} memudahkan perpindahan antar lingkungan, seperti Lokal dan Staging, tanpa mengubah request. Pada arsitektur MVC yang sudah berjalan, resource baru berupa registrasi pengguna dapat ditambahkan hanya dengan membuat tabel users, Model User, Controller AuthController, dan satu pemetaan pada Router, tanpa mengubah resource yang sudah ada.

Pada proses registrasi, kata sandi disimpan menggunakan password_hash() dengan PASSWORD_DEFAULT sehingga yang tersimpan di database adalah hash, bukan teks asli, dan validasi format email dengan filter_var() mencegah data yang tidak valid tersimpan. Penggunaan kode status HTTP yang tepat, yaitu 201 untuk registrasi berhasil, 400 untuk field kosong, 422 untuk format email tidak valid, dan 405 untuk method yang salah, membuat hasil pengujian mudah dibaca dan dibedakan. Contoh response sukses dan gagal yang disimpan pada setiap request juga menjadi dokumentasi awal API yang akan dipakai sebagai dasar pengembangan fitur login pada acara berikutnya.